

Практика 4: [Опционально] Расширение системы

Table of contents

Практика 4: [Опционально] Расширение системы	2
Статус практики	2
Что можно выбрать	2
Протокол отчёта	3
1. Проблема	3
2. Аналоги и варианты решения	4
3. Выбранная схема для вашей системы	4
4. Реальные структуры данных	5
5. Цена решения и ограничения	5
6. Минимальный план внедрения	5
Что сдаётся	6
Пример 1. Структурированные логи для интернет-магазина	6
1. Проблема	6
2. Аналоги	6
3. Схема внедрения	7
4. Структуры данных	7
5. Цена и ограничения	8
Пример 2. Геопоиск для сервиса доставки еды	8
1. Проблема	8
2. Аналоги	8
3. Схема внедрения	9
4. Структуры данных	9
5. Цена и ограничения	10
Пример 3. Аналитическая витрина для интернет-магазина	10
1. Проблема	10
2. Аналоги	11
3. Схема внедрения	11
4. Структуры данных	11
5. Цена и ограничения	13
Зачем это нужно	13
Критерии оценки	13

Практика 4: [Опционально] Расширение системы

Статус практики

Практика дополнительная. Здесь вы исследуете, как расширить систему из предыдущих практик одной новой подсистемой: поиском, геоданными, наблюдаемостью, безопасностью, операционным компонентом, аналитической витриной или другим подходящим модулем.

Код писать не обязательно. Достаточно показать схему, провести небольшой обзор возможных решений и объяснить, как выбранное решение можно применить в вашей задаче.

После первых трёх практик у вас уже есть основа проекта:

1. В Практике 1 вы спроектировали продукт, пользовательский сценарий, API и архитектуру.
2. В Практике 2 вы реализовали приложение со слоями, портами и репозиториями.
3. В Практике 3 вы спроектировали OLTP-схему, написали SQL и посмотрели планы выполнения запросов.

В Практике 4 нужно взять эту основу и спроектировать одну подсистему на выбор. Подойдут, например:

- структурированные логи;
- геописк;
- полнотекстовый поиск;
- векторный поиск;
- аналитическая витрина;
- ограничение частоты запросов;
- аутентификация и авторизация;
- наблюдаемость;
- кэш;
- очередь;
- фоновая обработка;
- другой компонент, который подходит вашему проекту.

Выбор должен быть связан с задачей. В отчёте нужно объяснить проблему, посмотреть аналоги, выбрать схему внедрения для своей системы и показать реальные структуры данных. При такой свободе особенно важна конкретика: привязывайте решение к одному обработчику API, сценарию, таблице, событию или потоку данных из вашего проекта.

Что можно выбрать

Выберите один вариант расширения:

- **Структурированные логи и наблюдаемость:** JSON-логи, идентификатор корреляции или трассировки, метрики, трассы, сбор в Loki / Elasticsearch / OpenTelemetry.

- **Геопоиск:** ближайшие водители, рестораны в зоне доставки, пункты выдачи рядом с пользователем, поиск по радиусу или по полигону.
- **Аналитика и витрина:** витрина продаж, событийная аналитика, ClickHouse / DuckDB / BigQuery, загрузка данных из OLTP.
- **Полнотекстовый или префиксный поиск:** поиск товаров, постов, врачей, курсов, подсказки при вводе.
- **Векторный поиск:** похожие товары, похожие посты, поиск по смыслу, рекомендации.
- **Ограничение нагрузки:** лимит запросов, антиспам, защита дорогих обработчиков API, очереди для тяжёлых задач.
- **Безопасность:** аутентификация, авторизация, роли, права, журнал аудита.
- **Кэширование:** Redis, кэш в памяти, сброс кэша, TTL, защита от одновременного пересчёта одного ключа.
- **Асинхронная обработка:** очередь задач, таблица исходящих событий, отправка уведомлений, пересчёт агрегатов.

Можно предложить свой вариант, если он продолжает сценарий из практик 1-3.

Чтобы тема не расплывалась, зафиксируйте границы работы:

- какой сценарий проекта вы улучшаете;
- какой компонент добавляете;
- где он стоит на схеме;
- какие данные читает и пишет;
- как поймёте, что решение подходит.

Протокол отчёта

Чтобы отчёты было проще читать и сравнивать, придерживайтесь одной структуры.

1. Проблема

Опишите конкретную проблему в вашей системе после Практики 3.

Лучше писать через пользовательский или операционный сценарий:

- оператор поддержки не может найти, почему заказ перешёл в ошибочный статус;
- пользователь такси ждёт список ближайших водителей меньше 300 мс;
- менеджер хочет видеть выручку по дням без тяжёлых `GROUP BY` в OLTP-базе;
- поиск по `ILIKE '%iphone%'` на 500k товаров занимает секунды;
- дорогой обработчик API можно вызвать 1000 раз в минуту и перегрузить БД.

В разделе укажите:

- какой обработчик API, сценарий или таблицы затрагиваются;
- сколько данных или запросов ожидается;
- какая задержка, точность, надёжность или стоимость считается приемлемой;
- где текущая система из практик 1-3 упирается в ограничение.

2. Аналоги и варианты решения

Рассмотрите 2-3 возможных подхода. Для каждого подхода кратко укажите:

- какую структуру данных или архитектурную схему он использует;
- какие продукты или библиотеки подходят;
- чем решение удобно;
- какую цену придётся заплатить: память, сложность, задержка записи, временно несогласованные данные, поддержка инфраструктуры.

Примеры сравнения:

Проблема	Вариант 1	Вариант 2	Вариант 3
Геопоиск	PostGIS GiST	Redis GEO	геохеш / квадродерево в приложении
Аналитика	ClickHouse	DuckDB + Parquet	материализованные представления в PostgreSQL
Логи	JSON-логи + Loki	ELK / OpenSearch	OpenTelemetry Col- lector
Поиск	PostgreSQL GIN	Elasticsearch	префиксное дерево / автодополнение в Redis
Ограничение частоты запросов	ведро токенов	счётчик скользящем окне	в фиксированное окно

3. Выбранная схема для вашей системы

Выберите один вариант и объясните, почему он подходит вашему проекту.

Покажите:

- где новый компонент появляется на архитектурной схеме;
- какие существующие сценарии, обработчики, репозитории или таблицы меняются;
- какие новые порты / интерфейсы появляются;
- какие реализации этих портов используются;
- синхронно или асинхронно работает компонент;
- что происходит при ошибке компонента.

Если вы придерживаетесь чистой архитектуры, подключайте новый компонент через порт. Сценарий должен зависеть от интерфейса, а конкретный клиент ClickHouse, PostGIS, Loki, Redis или Kafka остаётся в инфраструктурном слое.

Пример формата:

Слой	Новый компонент	Тип	С чем связан
Сценарии	CreateOrderUseCase	существующий	вызывает AuditLogPort после смены статуса
Порты	AuditLogPort	интерфейс	контракт записи события аудита
Инфраструктура	LokiAuditLogWriter	реализация	пишет JSON в стандартный вывод или Loki
Хранилище	audit_events	таблица или поток	хранит события для расследований

4. Реальные структуры данных

Покажите конкретные структуры, по которым можно понять, как решение будет работать.

В зависимости от темы подойдут:

- SQL DDL таблиц и индексов;
- JSON-схема события или лога;
- ClickHouse DDL;
- GraphQL / REST / gRPC-контракт нового обработчика API;
- структура ключей Redis;
- формат сообщения в очереди;
- интерфейсы портов;
- параметры индекса: GIN, GiST, HNSW, M, ef_search, TTL, размер корзины.

Минимум: 2-3 реальные структуры данных. Для каждой напишите, кто её пишет, кто читает и как она обновляется.

5. Цена решения и ограничения

Опишите компромиссы:

- сколько памяти или диска занимает структура;
- как меняется скорость записи;
- есть ли задержка между OLTP и новым компонентом;
- как обрабатываются дубли, ретраи и ошибки;
- какие данные могут устареть;
- какие метрики покажут, что решение работает.

6. Минимальный план внедрения

Опишите 3-5 шагов:

1. Добавить порт / интерфейс.
2. Добавить реализацию и структуру хранения.

3. Подключить компонент к одному сценарию.
4. Добавить тестовый сценарий или демонстрационный запрос.
5. Измерить или оценить ключевую метрику.

Код писать не обязательно. Можно ограничиться проектированием и исследованием решения. В таком случае отчёт должен быть достаточно конкретным, чтобы по нему можно было реализовать компонент без дополнительного обсуждения.

Что сдаётся

1. Документ `markdown` / `pdf` / `qmd` по протоколу выше.
2. Обновлённая архитектурная схема с новым компонентом.
3. Реальные структуры данных: DDL, JSON, интерфейсы, ключи, сообщения, конфигурация индекса.
4. Один демонстрационный сценарий: SQL-запрос, API-вызов, пример лога, пример события, пример панели или пример ответа нового обработчика API.
5. Код не обязателен. Если он есть, добавьте ссылки на файлы и инструкцию запуска.

Пример 1. Структурированные логи для интернет-магазина

1. Проблема

В интернет-магазине есть обработчик `POST /orders`. Иногда заказ получает статус `failed`, и команде нужно быстро понять, где произошла ошибка: при проверке корзины, резервировании товара, записи заказа или отправке уведомления.

Требование: для каждого заказа можно за 1-2 минуты найти цепочку событий по `order_id` или `trace_id`. Логи должны быть машинно-читаемыми, чтобы их можно было фильтровать в Loki или Elasticsearch.

2. Аналоги

Вариант	Что даёт	Цена
Обычный текстовый лог	просто внедрить	трудно фильтровать, нет стабильной схемы
JSON-логи + Loki	недорогой сбор логов, фильтр по меткам	нужно следить за кардинальностью меток
Трассы OpenTelemetry	видно дерево вызовов и задержки	больше инфраструктуры и инструментирования

Выбор: JSON-логи + `trace_id`. Для такой задачи достаточно хорошо искать события заказа и видеть причину ошибки. Полную распределённую трассировку можно добавить позже, когда появится несколько сервисов и станет важен путь запроса между ними.

3. Схема внедрения

Слой	Компонент	Тип	Назначение
Сценарии	CreateOrderUseCase	существующий	пишет события order_created, stock_reserved, order_failed
Порты	StructuredLogger	интерфейс	принимает типизированное событие
Инфраструктура	JsonLogger	реализация	сериализует событие в JSON
Эксплуатация	Loki / OpenSearch	хранилище	индексирует логи по времени и меткам

Порт:

```
type StructuredLogger interface {  
    Info(ctx context.Context, event LogEvent)  
    Error(ctx context.Context, event LogEvent)  
}
```

4. Структуры данных

```
{  
    "timestamp": "2026-05-13T12:30:15.123Z",  
    "level": "ERROR",  
    "service": "shop-api",  
    "trace_id": "9f7c2ale",  
    "user_id": 123,  
    "order_id": 987,  
    "event": "order_failed",  
    "reason": "insufficient_stock",  
    "product_id": 55,  
    "requested": 3,  
    "available": 1,  
    "duration_ms": 42  
}
```

```
type LogEvent struct {  
    Event    string
```

```

    UserID      *int64
    OrderID     *int64
    ProductID   *int64
    Reason      string
    DurationMs  int64
    Fields      map[string]any
}

```

Метки Loki:

```

labels:
  service: shop-api
  level: error
  event: order_failed

```

`user_id` и `order_id` остаются в JSON-поле. В метки берите поля с небольшим числом значений, иначе индекс станет тяжёлым.

5. Цена и ограничения

Запись лога добавляет небольшую нагрузку на процессор из-за JSON-сериализации. При большом трафике отладочные логи лучше включать точечно. Если Loki недоступен, приложение продолжает работать: лог уходит в `stdout`, а сборщик заберёт его позже.

Ключевые метрики: количество `order_failed`, доля ошибок по обработчику API, `p95 duration_ms`, время поиска инцидента по `trace_id`.

Пример 2. Геопоиск для сервиса доставки еды

1. Проблема

Пользователь вводит адрес, а система должна показать рестораны, которые доставляют в эту точку. В OLTP-схеме из Практики 3 рестораны уже есть, но обычный B-tree по широте и долготе плохо подходит для запросов по зоне доставки или радиусу.

Требование: `p95` меньше 200 мс при 100k ресторанов и частоте 50 запросов в секунду.

2. Аналоги

Вариант	Что даёт	Цена
PostGIS + GiST	точные геометрические запросы, SQL-интеграция	нужно расширение PostGIS и понимание SRID
Redis GEO	быстрый поиск по радиусу в памяти	ограничен точками, сложнее с полигонами доставки

Вариант	Что даёт	Цена
Геохеш / квадродерево	можно реализовать в приложении	больше собственного кода и краевых случаев

Выбор: PostGIS + GiST. В проекте уже используется PostgreSQL, а зоны доставки удобно хранить как геометрию: так проще описывать полигоны и проверять попадание адреса.

3. Схема внедрения

Слой	Компонент	Тип	Назначение
Сценарии	FindAvailableRestaurants	интерфейс или новый	ищет рестораны для адреса
Порты	RestaurantGeoRepository	интерфейс	контракт геопоиска
Инфраструктура	PostgisRestaurantGeoRepository	реализация	выполняет SQL с ST_Contains / ST_DWithin
Хранилище	restaurant_delivery_zones	таблица	хранит полигоны доставки

Порт:

```

type RestaurantGeoRepository interface {
    FindByPoint(ctx context.Context, point GeoPoint) ([]Restaurant, error)
}

type GeoPoint struct {
    Lat float64
    Lon float64
}

```

4. Структуры данных

```

CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TABLE restaurant_delivery_zones (
    restaurant_id BIGINT PRIMARY KEY REFERENCES restaurants(id),
    zone geometry(Polygon, 4326) NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_restaurant_delivery_zones_gist

```

```
ON restaurant_delivery_zones
USING GIST (zone);
```

```
SELECT r.id, r.name, r.rating
FROM restaurants r
JOIN restaurant_delivery_zones z ON z.restaurant_id = r.id
WHERE ST_Contains(
    z.zone,
    ST_SetSRID(ST_MakePoint($1, $2), 4326)
)
ORDER BY r.rating DESC
LIMIT 50;
```

Если нужен поиск ближайших курьеров:

```
CREATE TABLE courier_locations (
    courier_id BIGINT PRIMARY KEY,
    location geography(Point, 4326) NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL
);

CREATE INDEX idx_courier_locations_gist
ON courier_locations
USING GIST (location);
```

5. Цена и ограничения

GiST-индекс занимает место на диске и замедляет обновление зон доставки. Для ресторанов подходит: зоны меняются редко. Координаты курьеров обновляются часто, и для них может понадобиться Redis GEO или отдельное хранилище в памяти.

Метрики: p95 геозапроса, количество ресторанов-кандидатов, доля пустых выдач, частота обновления геоданных.

Пример 3. Аналитическая витрина для интернет-магазина

1. Проблема

Менеджер хочет видеть выручку по дням, топ товаров и средний чек. Если считать отчёты прямо в PostgreSQL по таблицам `orders` и `order_items`, аналитические `GROUP BY` начинают мешать OLTP-запросам оформления заказа.

Требование: отчёты за последние 90 дней открываются меньше чем за 1 секунду, а OLTP-база остаётся под пользовательскую нагрузку.

2. Аналоги

Вариант	Что даёт	Цена
Материализованные представления в Post-greSQL	минимум инфраструктуры	нагрузка остаётся в OLTP, сложнее масштабировать аналитику
ClickHouse	быстрые агрегации по колонкам	отдельная БД и процесс загрузки
DuckDB + Parquet	простая локальная аналитика	хуже подходит для многопользовательских панелей

Выбор: ClickHouse. Он хорошо подходит для повторяемых отчётов по большим таблицам, а аналитическая нагрузка уходит из OLTP-базы.

3. Схема внедрения

Слой	Компонент	Тип	Назначение
Сценарии	CreateOrderUseCase	существующий	после успешного заказа публикует событие
Порты	OrderEventPublisher	интерфейс	контракт публикации события
Инфраструктура	OutboxOrderEventPublisher	реализация	пишет событие в таблицу исходящих событий
Загрузка данных	orders_to_clickhouse_job	фоновый процесс	переносит события в ClickHouse
Аналитическая витрина	orders_fact	таблица	хранит строки фактов для отчётов

4. Структуры данных

Таблица исходящих событий в PostgreSQL:

```
CREATE TABLE outbox_events (  
  id BIGSERIAL PRIMARY KEY,  
  event_type TEXT NOT NULL,  
  aggregate_id BIGINT NOT NULL,  
  payload JSONB NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```

        processed_at TIMESTAMPTZ
    );

    CREATE INDEX idx_outbox_unprocessed
    ON outbox_events (created_at)
    WHERE processed_at IS NULL;

```

Событие:

```

{
  "event_type": "order_paid",
  "order_id": 987,
  "user_id": 123,
  "paid_at": "2026-05-13T12:30:15Z",
  "items": [
    {"product_id": 55, "category_id": 7, "quantity": 2, "price": "1990.00"}
  ]
}

```

ClickHouse:

```

CREATE TABLE orders_fact (
    order_id UInt64,
    user_id UInt64,
    product_id UInt64,
    category_id UInt64,
    quantity UInt16,
    price Decimal(12, 2),
    paid_at DateTime
)
ENGINE = MergeTree()
PARTITION BY toYYYYMM(paid_at)
ORDER BY (product_id, paid_at, order_id);

```

Отчёт:

```

SELECT
    toDate(paid_at) AS day,
    sum(quantity * price) AS revenue,
    uniqExact(order_id) AS orders_count
FROM orders_fact
WHERE paid_at >= today() - INTERVAL 90 DAY

```

```
GROUP BY day  
ORDER BY day;
```

5. Цена и ограничения

Данные в аналитической витрине появляются с задержкой: например, раз в минуту или раз в час. Загрузку нужно сделать идемпотентной, чтобы повторная обработка исходящего события проходила без дублей. Источником истины остаётся OLTP-база, а корректировки заказов приходят в витрину отдельными событиями.

Метрики: задержка между `created_at` и загрузкой в ClickHouse, количество необработанных исходящих событий, время выполнения отчётов, размер партиций.

Зачем это нужно

В реальной работе вам вряд ли понадобится всё, что есть в курсе. Кто-то никогда не будет вручную создавать индекс. Кто-то не полезет во внутреннее устройство аналитических хранилищ, шардирование и партиционирование. Кому-то не пригодятся эмбединги, геодеревья или настройка SSO.

Но почти наверняка вы столкнётесь хотя бы с одной такой темой. В этот момент хочется понимать инструмент глубже, чем на уровне `SELECT id FROM users`. Легко годами пользоваться базой, очередью, кэшем или поиском и не интересоваться, что происходит внутри. Задачи закрываются, зарплата приходит, но профессиональный рост быстро упирается в потолок.

Эта практика проверяет именно такой навык: заметить проблему, сравнить варианты, выбрать решение и набросать план внедрения. В реальной работе с помощью LLM такой черновик можно собрать за дни, а не за месяцы. Главное: поставить себе такую задачу и довести рассуждение до конкретной схемы.

Критерии оценки

- Проблема сформулирована через конкретный сценарий вашей системы.
- Рассмотрены 2-3 аналога.
- Выбранная схема встроена в архитектуру из практик 1-3.
- Показаны реальные структуры данных и контракты.
- Описаны ограничения, стоимость и метрики.
- По отчёту понятно, как внедрять компонент.